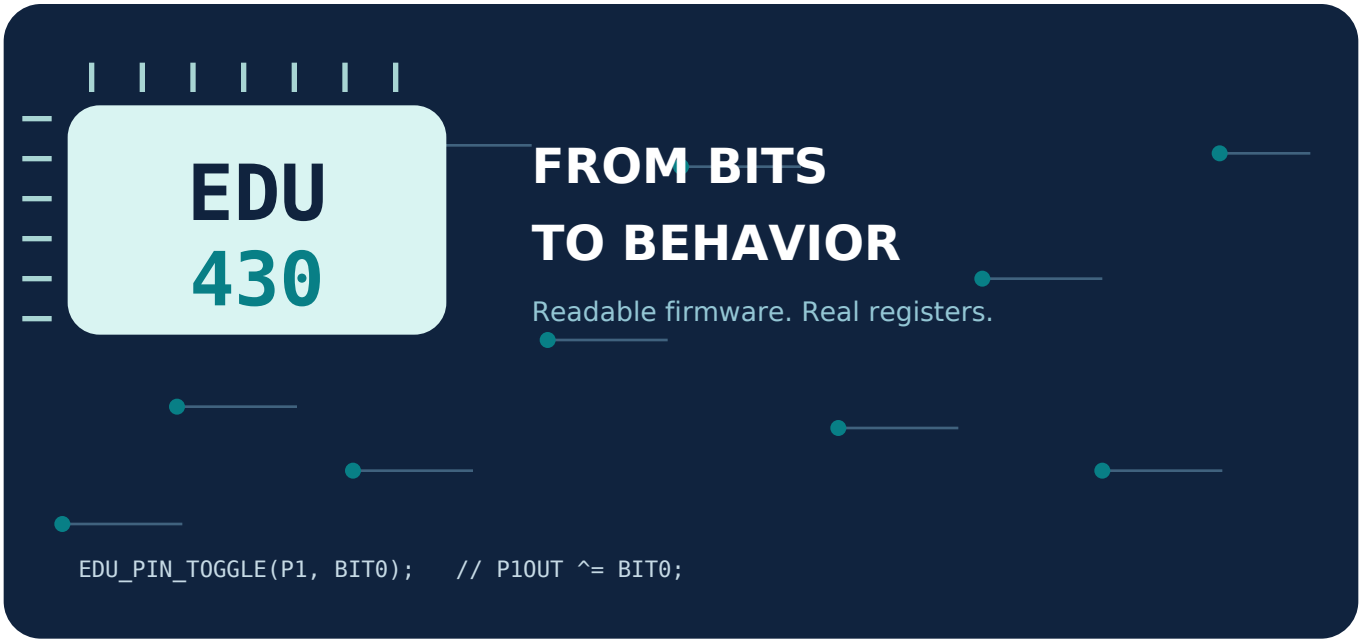


Edu430 FwLib v0.2

User / Programmer / Instructor Guide

An expert guide to the MSP430G2553 educational firmware library: quick starts, complete API inventory, register-to-macro explanations, bus protocols, classroom investigations, and a realistic validation plan.

RELEASE	TARGET	COMPATIBILITY
v0.2 • Teaching preview	MSP430G2553 16 KB flash / 512 B RAM	MSP-EXP430G2 / MSP-EXP430G2ET

Prepared: 22 October 2026 **Distribution:** GPL-3.0 within the learning curriculum **Status:** Host-tested; hardware bus validation outstanding

Source: RandoSY / nugget-physical-computing / MSP430G2553 Learning Labs / fwlib

At a glance

The library gives newcomers meaningful commands while keeping register operations visible. It is neither an Arduino compatibility layer nor a real-time operating system. It is a curated set of header-only macros and short inline functions for the MSP430G2553.

63	38	14	8
Macro definitions (including constants and helpers)	Inline functions	Header files	CCS example folders

Read this manual by role

If you are…	Start here	Your immediate objective
New to the MSP430	Chapters 1–3	Blink an LED, use the button, read registers.
Writing or reviewing C firmware	Chapters 4–7	Select appropriate modules and use accurate APIs.
Teaching a laboratory	Chapters 8–10	Run experiments, collect evidence and assess comprehension.
Validating a hardware platform	Chapters 6, 11	Confirm wiring, bus transactions and failure recovery.

Release qualification Host-side syntax checking covered the eight Edu430 examples, with three deterministic tests. That does **not** establish successful CCS import, real MSP430 compilation, timing compliance, correct I²C/SPI waveforms, or general sensor compatibility.

Safety in one minute

- Operate MSP430 GPIO at its 3.3 V domain. Never wire 5 V Echo or 5 V pull-ups directly to the target inputs.
- Both hardware SPI and hardware I²C use USCI_B0; P1.6 also serves onboard LED B. These functions compete for the same pins.
- With 1 MHz SMCLK, the provided UART uses 9600 8-N-1. Reconfigure the UART if changing clock frequency.
- The scheduler and TA0 PWM cannot both claim Timer_A0 in the same program without redesign.

Contents

At a glance 2

Read this manual by role . . . 2

Safety in one minute . . . 2

Contents 3

1 Philosophy and architecture 7

Four layers; one electrical reality . . . 7

Where Edu430 draws the abstraction boundary . . . 7

Why the EDU prefix matters . . . 7

Non-goals . . . 7

2 Installation and five-minute quick start 8

Prerequisites . . . 8

Choosing the project . . . 8

Minimal register-transparent LED demonstration . . . 8

Debugging sequence . . . 8

Backchannel UART . . . 9

3 A tour of the MSP430G2553 hardware 9

Memory-mapped peripherals . . . 9

Board pin and resource matrix . . . 10

Clocks and real time . . . 10

4 API orientation: macros or functions? 10

The naming convention . . . 10

The canonical bit helpers . . . 11
GPIO token-pasting (not runtime port selection) . . . 11
Multi-operation macros . . . 11
Functions give back explicit success/failure . . . 11

5 Peripheral programming handbook 11

5.1 GPIO and pin configuration . . . 11
5.2 Port interrupts . . . 12
5.3 Timer_A and cooperative scheduling . . . 12
5.4 PWM waveform control . . . 12
5.5 ADC10 measurement . . . 13
5.6 UART serial backchannel . . . 13
5.7 SPI master: exchange bytes . . . 13
5.8 Hardware I²C: USCI_B0 . . . 13
5.9 Software I²C: GPIO teaches the protocol . . . 14
5.10 Comparator_A+ and button filtering . . . 14

6 Bus engineering and failure analysis 15

SPI versus I²C . . . 15
A defensible measurement and debug workflow . . . 15
Potential release limitations worth teaching . . . 15
How to write a safe sensor adapter . . . 15

7 Source organization and reference practices 16

The library layout . . . 16
Why the examples copy their own headers . . . 16

Safe coding rules . . . 16

8 Instructor handbook: inquiry-based STEM 16

Eight suggested sessions (50–75 minutes each) . . . 17

Lab record template . . . 18

Suggested assessment rubric . . . 18

EnergyTrace inquiry extension . . . 18

9 Programmers: API index 18

Bit and token helpers [edu430_bits.h] . . . 19

Digital I/O / interrupts [edu430_gpio.h] . . . 20

Board-level conveniences [edu430.h] . . . 20

Clock / watchdog / sleep [edu430_system.h] . . . 21

Timer_A [edu430_timer.h] . . . 21

Cooperative scheduling [edu430_scheduler.h] . . . 21

ADC10 [edu430_adc.h] . . . 22

UART [edu430_uart.h] . . . 22

SPI [edu430_spi.h] . . . 22

Hardware I²C [edu430_i2c_hw.h] . . . 23

Software I²C [edu430_i2c_soft.h] . . . 23

PWM [edu430_pwm.h] . . . 24

Comparator_A+ [edu430_comparator.h] . . . 24

Button events [edu430_button.h] . . . 24

10 Teacher-facing capstone projects 25

Capstone A • A temperature telemetry instrument . . . 25

Capstone B • A load-cell scale . . . 25
Capstone C • Two range sensors . . . 25
Capstone D • Scheduled instrument and power . . . 25

11 Testing, qualification and release engineering 25

What has been checked . . . 25
Recommended validation plan . . . 26
Release gating . . . 26

12 Troubleshooting and field checklist 27

13 Source lineage, further reading and glossary 27

Primary references . . . 27
Glossary . . . 28
Closing design principle . . . 28

For an instructional reading sequence, the manual begins with practical operation, then moves down toward register-level C and protocol details. The API appendix is deliberately exhaustive for the v0.2 source.

1 Philosophy and architecture

Our back-row design standard is practical: a learner should make the hardware do something observable before being required to understand every register bit. The very same experiment then becomes a vehicle for learning those bits.

Four layers; one electrical reality

Layer	Expression	Interpretation
Application / board	`EDU_LED_A_TOGGLE();`	Toggle a visible LED on this LaunchPad.
Peripheral abstraction	`EDU_PIN_TOGGLE(P1, BIT0);`	Toggle pin P1.0 of Port 1.
Bit operation	`EDU_TOGGLE(P1OUT, BIT0);`	Exclusive-OR the output latch bit.
Hardware register C	`P1OUT ^= BIT0;`	Change the output latch and electrical state.

Instructional invariant Every public operation must be traceable to a register expression, peripheral state machine, or short named function. **The convenience layer is removable scaffolding, not hidden magic.**

Where Edu430 draws the abstraction boundary

- **Macro:** clear, set or test bits; choose a port or Timer_A register; call an already-defined task.
- **Inline function:** ADC conversion, UART transmission, SPI transaction, I²C bus sequencing, debounce or elapsed-time logic.
- **Board layer:** identify LED A, LED B and button without hardcoding colors; more complex instruments should add a custom board map.
- **Application:** decide when and why an action occurs, how results are interpreted, and how errors are reported.

Why the EDU prefix matters

The `EDU_` namespace identifies the teaching API and prevents confusion with TI-defined device symbols such as `BIT0`, `TA0CTL`, `ADC10CTL0` and `UCB0TXBUF`. Lower-case `edu_` identifies the small C functions that implement multi-step operations. These names are intentionally different from the historic `MSP_` macro layer and the upstream Piconomix `px_` style.

Non-goals

No dynamic allocation, no scheduler stacks, no RTOS, no transparent pin remapping, no universal device abstraction, no flash-memory write convenience, and no assumption that low-level peripheral operations are safe merely because they compile.

2 Installation and five-minute quick start

Prerequisites

- A LaunchPad with an MSP430G2553 fitted, USB cable, supported Code Composer Studio with MSP430 device support.
- For serial work, a Windows COM terminal configured for 9600 baud, 8 data bits, no parity, one stop bit.
- For I²C, two pull-ups to 3.3 V and a voltage-compatible slave. For SPI, a 3.3 V device with an independently wired chip-select line.

Choosing the project

From the ZIP, select one of `examples/01-port-macros` through `examples/08-compare-macro-with-register`. Each example has its own `main.c`, local copies of Edu430 headers, and a CCS `.projectspec` import candidate. Attempt Project → Import CCS Projects if your CCS version recognizes the ProjectSpec; otherwise create an MSP430G2553 C project and add the desired example files.

> **Build discipline** Compile **one** example at a time. Each example includes its own `main()` and its own local headers. Avoid adding multiple example directories to a single CCS project.

Minimal register-transparent LED demonstration

```
#include "edu430.h"

int main(void)
{
    EDU_WATCHDOG_STOP();
    if (!edu_clock_1mhz()) { while (1) {} }

    EDU_GPIO_OUTPUT_LOW(P1, BIT0);  // P1OUT &= ~BIT0;
                                    // P1DIR |= BIT0;
    while (1)
    {
        EDU_PIN_TOGGLE(P1, BIT0);  // P1OUT ^= BIT0;
        __delay_cycles(250000);    // 250 ms at 1 MHz
    }
}
```

Students should verify changes to P1DIR and P1OUT in the CCS Registers view. The clock routine checks factory calibration before applying it.

Debugging sequence

- Build. Confirm compiler target MSP430G2553. Select the correct debugger for G2 or G2ET.
- Run the example. Observe physical behavior and port registers in CCS.
- Break on the `EDU_PIN_TOGGLE` statement, then find `EDU_PIN_TOGGLE` in `edu430_gpio.h`.
- Expand through `EDU_TOGGLE` in `edu430_bits.h`. Explain the `^=` operator.
- Re-run with the macro replaced by the raw register statement and compare results.

Backchannel UART

```
EDU_WATCHDOG_STOP();
(void)edu_clock_1mhz();
edu_uart_9600_init();
(void)edu_uart_text("Hello from Edu430!\r\n", 100000UL);
```

USB serial passes through the LaunchPad debugger, then uses P1.1 as MCU receive and P1.2 as transmit. On the original G2, check HW/SW UART jumper orientation; on the G2ET, check UART jumper routing. COM number varies with the host.

3 A tour of the MSP430G2553 hardware

Memory-mapped peripherals

MSP430 registers are memory-mapped storage locations. Expressions such as `P1DIR |= BIT0` change a real control bit rather than a C-only variable. Peripheral selection, interrupt flags, prescalers and clock sources all require explicit register setup.

Block	Relevant symbols	Edu430 entry points
GPIO	P1DIR, P1OUT, P1IN, P1REN, P1SEL, P1SEL2	EDU_GPIO_*, EDU_PIN_*
Clock and watchdog	DCOCTL, BCSCTL1, BCSCTL2, WDTCTL	edu_clock_1mhz(), EDU_WATCHDOG_*
Timer_A	TA0CTL, TA0CCR0, TA0CCTL0	EDU_TIMER_UP, EDU_RUN_EVERY
ADC10	ADC10CTL0, ADC10CTL1, ADC10MEM	edu_adc10_init_a4(), edu_adc10_sample()
UART (USCI_A0)	UCA0CTL1, UCA0BR0, UCA0TXBUF, IFG2	edu_uart_*
SPI/I²C (USCI_B0)	UCB0CTL0, UCB0CTL1, UCB0TXBUF	edu_spi_*, edu_i2c_*
Comparator_A+	CACTL1, CACTL2	EDU_COMP_*

Board pin and resource matrix

Signal / purpose	MCU pad	Concurrent-use constraint
LED A	P1.0	Shares ADC channel A0; GPIO/ADC selection matters.
LED B	P1.6	Also USCI_B0 MISO (SPI) / SCL (I²C) and TA0.1 PWM.
Button	P1.3	Input with pull-up, active-low; P1 interrupt possible.
UART RX / TX	P1.1 / P1.2	Backchannel jumper routing on board; USART pin selection.
ADC voltmeter	P1.4 / A4	External signal must stay within supply rails.
SPI CLK / MISO / MOSI	P1.5 / P1.6 / P1.7	USCI_B0 cannot be used as hardware I²C simultaneously.
Hardware I²C SCL / SDA	P1.6 / P1.7	External 3.3 V pull-ups; conflicts with LED B and SPI.
Software I²C SCL / SDA	P2.0 / P2.1	GPIO, external pull-ups; P2.1 may have board-specific RGB-LED loading.
TA0 1 ms scheduler	Timer_A0 CCR0	Cannot independently use TA0 for PWM as configured.

Original G2 versus G2ET The MCU register map is the same if both boards actually contain the G2553, but the USB debugger and external wiring differ. The newer G2ET adds EnergyTrace hardware. Do not assume onboard LED colors or jumper layout from the older board.

Clocks and real time

The stock v0.2 functions intentionally assume a factory-calibrated 1 MHz DCO in their 9600-baud UART, 1 kHz PWM and one-millisecond Timer_A examples. If you change clocks, review **every** serial divider, timer count and instruction-cycle delay. A fixed poll-count timeout is not the same as a wall-clock guarantee.

4 API orientation: macros or functions?

The naming convention

Names beginning with uppercase `EDU_` are preprocessor definitions. The same headers use lower-case `edu_` for `static inline` functions. TI peripheral constants remain uppercase but do not use `EDU_`. This distinction enables students to locate each implementation in seconds.

The canonical bit helpers

```
EDU_SET(P1OUT, BIT0);          // (P1OUT |= BIT0)
EDU_CLEAR(P1OUT, BIT0);        // (P1OUT &= ~BIT0)
EDU_TOGGLE(P1OUT, BIT0);       // (P1OUT ^= BIT0)
if (EDU_ALL_SET(P1IN, BIT3)) {  // (P1IN & BIT3) == BIT3
    // Button is high (normally released with pull-up)
}
```

GPIO token-pasting (not runtime port selection)

```
// Written by learner:
EDU_GPIO_OUTPUT(P1, BIT0);

// Port token is expanded during preprocessing:
P1SEL  &= ~BIT0;
P1SEL2 &= ~BIT0;
P1DIR  |=  BIT0;

// 'P1' is a literal token, not a runtime variable.
```

Multi-operation macros

Macros such as `EDU_GPIO_PULLUP` use `do { ... } while(0)` so they behave like a single C statement in an if/else. Peripheral macros may modify more than one register, so consult the header and the atlas before using them during interrupt-sensitive operations.

Functions give back explicit success/failure

```
uint8_t received;
if (!edu_spi_transfer(0x9F, &received, 100000UL))
{
    // Timeout or peripheral was not ready.
    // Check wiring, mode, clocks and power.
}
```

Return conventions In this release, bus/status functions generally return 1 for success and 0 for failure. These are **not** rich diagnostic codes. The source is authoritative for individual functions; some initialization functions return void and SPI read output is through a pointer.

5 Peripheral programming handbook

5.1 GPIO and pin configuration

Configure an output latch before enabling drive to reduce startup glitches. `EDU_GPIO_OUTPUT_LOW(P1, BIT0)` does this in the correct order and selects GPIO functionality. Internal pull-ups are appropriate for the active-low P1.3 button.

```
EDU_GPIO_OUTPUT_LOW(P1, BIT0);   // known LOW then output
EDU_GPIO_PULLUP(P1, BIT3);      // input, resistor, HIGH idle
if (EDU_PIN_LOW(P1, BIT3))       // active-low switch
    EDU_PIN_TOGGLE(P1, BIT0);
```

5.2 Port interrupts

Port 1 and Port 2 provide edge interrupts. Clear the pending IFG after selecting the desired edge; enable the relevant port vector and global interrupts. Port 3 interrupt register macros must not be used with a device/package that lacks P3IE/P3IES/P3IFG.

```
EDU_GPIO_IRQ_DISABLE(P1, BIT3);
EDU_GPIO_IRQ_FALLING(P1, BIT3);
EDU_GPIO_IRQ_CLEAR(P1, BIT3);
EDU_GPIO_IRQ_ENABLE(P1, BIT3);
EDU_IRQ_ON();
// Supply a proper PORT1_VECTOR ISR in the application.
```

5.3 Timer_A and cooperative scheduling

A Timer_A CCR0 interrupt supplies the timebase; it does not execute each job. The application sets up the timer, increments a volatile millisecond counter in the ISR, and calls `EDU_RUN_EVERY` in the main loop. Scheduling uses unsigned 16-bit subtraction and skips missed intervals by resetting the last-run timestamp to the current time.

```
volatile uint16_t tick_ms = 0;  // increment in Timer_A0 ISR
edu_job_t blink = {0u, 250u};

// Startup, with calibrated 1 MHz SMCLK:
EDU_TIMER_1MS_TA0();
EDU_IRQ_ON();

for (;;)
{
    uint16_t now = tick_ms;      // atomic on 16-bit MSP430
    EDU_RUN_EVERY(blink, now, toggle_led);
}
```

Scheduler bound Intervals of 1 to 32767 ms are supported by `edu_job_due()`. This is cooperative scheduling: slow UART output and long sensor waits delay other tasks. The Timer_A0 resource cannot also run the supplied PWM helper independently.

5.4 PWM waveform control

```
edu_pwm_ta0_p16_begin(500u);  // ~1 kHz, ~50% PWM
edu_pwm_ta0_p16_duty(250u);   // ~25% PWM
```

The stock driver clamps requested duty to 1..999 per mille; it intentionally excludes exact 0% and 100%. P1.6 is the output, so LED B and USCI_B0 serial functions are not simultaneously usable on that pad. Confirm waveform with a scope or logic analyzer before attaching a motor or power stage.

5.5 ADC10 measurement

```
uint16_t raw;
edu_adc10_init_a4();
if (edu_adc10_sample(&raw, 100000UL))
{
    // Approximate millivolts when AVCC is exactly 3300mV:
    uint32_t mv = (uint32_t)raw * 3300u / 1023u;
    // Actual VCC must be measured for accuracy.
}
```

The current driver selects A4/P1.4 and AVCC/AVSS reference. Measure the real supply for calibration. ADC conversion counts alone are not degrees Celsius, grams or centimeters; sensor models must be applied separately.

5.6 UART serial backchannel

```
edu_uart_9600_init();
(void)edu_uart_text("count,42\r\n", 100000UL);
uint8_t ch;
if (edu_uart_get(&ch)) { /* process one available byte */ }
```

UART send uses a bounded busy poll and is not suitable for execution inside a time-critical interrupt. Use a ring buffer and ISR-driven UART for more demanding asynchronous applications.

5.7 SPI master: exchange bytes

```
EDU_GPIO_OUTPUT_HIGH(P2, BIT0);  // manual chip select
edu_spi_begin(EDU_SPI_MODE0, 4u); // SMCLK / 4
EDU_PIN_OFF(P2, BIT0);
uint8_t rx;
(void)edu_spi_transfer(0x9Fu, &rx, 100000UL);
EDU_PIN_ON(P2, BIT0);
```

The example demonstrates the transaction mechanics, not a complete JEDEC driver. Many SPI devices require extra bytes after the command and have specific mode/maximum-frequency requirements. Chip-select is explicit, and both directions shift during every SPI byte.

5.8 Hardware I²C: USCI_B0

```
edu_i2c_begin();            // 1MHz / 20 = ~50kHz
uint8_t reg = 0x00u;
if (!edu_i2c_write(0x48u, &reg, 1u, 100000UL))
{
    // No ACK, busy bus or timeout; check the wiring.
}
```

The driver uses seven-bit addresses and supports basic reads/writes plus a read-register convenience routine. The USCI_B0 I²C state machine is subtle: address acknowledgments, START completion, final-byte handling and STOP timing must be checked against TI documentation and live bus traces. Treat the read path as provisional until qualified with actual devices.

5.9 Software I²C: GPIO teaches the protocol

```
if (edu_si2c_init())
{
    uint8_t id;
    if (edu_si2c_read_reg8(0x29u, 0x00u, &id))
    {
        // Received one byte from an 8-bit register address.
    }
}
```

The software driver releases lines instead of actively driving high, emulating open-drain behavior with P2DIR. It requires external pull-ups to 3.3 V. Clock stretching is polled; practical speed is implementation- and clock-dependent. A complete VL6180X driver needs 16-bit register-address transactions and an initialization sequence, which this v0.2 interface does not yet supply.

5.10 Comparator_A+ and button filtering

The comparator macros expose enable, output status, IRQ enable/disable, and interrupt flag clearing. Configuration of reference and pin routing is intentionally left explicit. `edu_button_pressed` uses consecutive sampled levels to generate a one-shot new-press event; it is called by a periodic scheduler rather than by a blocking delay.

```
edu_button_t button = {0u,0u,0u};
// Called every 10 milliseconds:
if (edu_button_pressed(&button, EDU_BUTTON_DOWN(), 3u))
{
    EDU_LED_A_TOGGLE();  // accepted press event
}
```

6 Bus engineering and failure analysis

SPI versus I²C

Topic	SPI	I²C
Clock/data	SCLK + MOSI + MISO	SCL + SDA, bidirectional
Addressing	GPIO chip select	7-bit address + ACK/NACK
Electrical	Push-pull, device-specific level	Open-drain, pull-ups mandatory
Common failure	Wrong mode / CS timing	No ACK / stuck bus / pull-up issue
G2553 peripheral	USCI_B0	USCI_B0
Firmware starting point	edu_spi_begin / transfer	edu_i2c_begin/write/read

A defensible measurement and debug workflow

- Verify VCC, GND, external pull-ups and chip-select at rest; only then recompile code.
- Measure SCL/SCLK and data pins with logic analyzer at the actual target clock rate.
- On I²C check address byte, ACK cycle, repeated START and STOP; check NACK and disconnected-device paths.
- On SPI confirm clock polarity and sampling phase against the peripheral datasheet; compare observed bytes with a known ID.
- Repeat with intentional faults: unplug module, missing pull-up, held SDA, late slave, and long cable.
- Report exact device, board revision, compiler version, settings and any applicable silicon errata.

Potential release limitations worth teaching

> **Implementation review points** The v0.2 hardware-I²C read routine and register-read sequence should undergo protocol review and oscilloscope qualification before they are relied upon for production or student grading. The supplied poll-count timeouts have no independently calibrated duration. Software I²C can interfere with P2.1 functionality on some board wiring.

How to write a safe sensor adapter

Separate the transport interface (I²C/SPI), the chip register map, and the science model (voltage, mass, distance, temperature). Keep calibration constants and units explicit, and expose errors rather than silently printing zero. In a school laboratory, reporting “no valid sample” is preferable to making up a measurement.

7 Source organization and reference practices

The library layout

```
Edu430_FwLib_v0_2/
    include/
        edu430.h             // umbrella header
        edu430_bits.h        // masks / register tokens
        edu430_gpio.h        // digital I/O and port IRQs
        edu430_system.h      // DCO, WDT, CPU power
        edu430_timer.h       // Timer_A
        edu430_scheduler.h   // cooperative jobs
        edu430_button.h      // debounce
        edu430_adc.h         // ADC10
        edu430_pwm.h         // Timer_A output
        edu430_comparator.h  // Comparator_A+
        edu430_uart.h        // USCI_A0
        edu430_spi.h         // USCI_B0 SPI
        edu430_i2c_hw.h      // USCI_B0 I2C
        edu430_i2c_soft.h    // bit-banged P2 I2C
    examples/                // 8 independent CCS demos
    docs/                    // original atlas and CCS notes
    tests/                   // deterministic host tests
```

Why the examples copy their own headers

An example folder is intended to be self-contained for classroom import. That simplifies file paths, but creates copies: when editing a header in `include/`, also update the example-local copies before declaring a new library release. Longer-term, a tested CCS library project or project-linked header directory would reduce duplication.

Safe coding rules

- Always parenthesize macro arguments; prefer `do { ... } while(0)` for statement-like macros.
- Do not pass expressions with increment, function calls or other side effects to register/mask macros.
- Use `volatile` for memory shared with an ISR; read 16-bit counters atomically on the 16-bit target.
- Name the clock source and divider in the lesson, not only the expected milliseconds.
- Prefer `static inline` over complex macros for operations needing loops, state or error returns.
- Do not erase info flash or overwrite factory DCO calibration data in beginner examples.

8 Instructor handbook: inquiry-based STEM

Instructional goal: students should be able to use a convenience command on day one and still explain the actual voltage, timing, or measurement it causes by the end of the sequence. Each laboratory follows a predict / build / observe / explain / extend cycle.

Eight suggested sessions (50–75 minutes each)

01 GPIO reveal

Question: What voltage is present at P1.0 when the LED changes?

Activity: Predict P1OUT/P1DIR, inspect CCS, measure with DMM. **Evidence:** Explain latch vs direction vs external LED.

02 UART console

Question: What establishes the bit interval at 9600 baud?

Activity: Send a recognizable string; alter SMCLK and diagnose corruption. **Evidence:** Relate baud rate to register divider.

03 ADC calibration

Question: Is a 10-bit reading a measurement of voltage?

Activity: Measure 0.5/1.5/2.5 V; graph ADC counts. **Evidence:** Slope, offset, quantization and VCC.

04 SPI memory

Question: How do devices know which bytes belong to them?

Activity: Read JEDEC ID; inspect CS/SCLK on scope. **Evidence:** Mode, framing, and identity vs command.

05 Soft I²C

Question: Can GPIO emulate a standard communication bus?

Activity: Scan addresses; trace START, ACK, STOP. **Evidence:** Open-drain, pull-up, address and failure.

06 HW I²C

Question: How does a peripheral engine differ from bit-banging?

Activity: Try a verified device; compare time and waveforms. **Evidence:** Peripheral ownership and state machine.

07 Cooperative time

Question: Can two programs run without threads?

Activity: Schedule LED and button jobs at different rates. **Evidence:** Polling, fairness, jitter and overruns.

08 Remove macros

Question: What is abstraction hiding?

Activity: Rewrite code as exact registers; compare behavior. **Evidence:** Translate intent ⇔ register ⇔ physics.

Lab record template

Student records	Minimum acceptable evidence
Prediction	Pin state, expected timing, address or numerical range.
Schematic	Power rails, signal labels, sensor part number, pull-ups/level shift.
Procedure	Which program/version, CCS build ID and settings.
Data	Measurements with units and uncertainties; failure observations too.
Explanation	Translate at least one `EDU_` command into register C.
Revision	One controlled change and the measured consequence.

Suggested assessment rubric

Criterion	Novice	Competent	Advanced
Physical model	Recognizes behavior	Predicts voltage/timing	Explains register-to-physics
Code reading	Uses commands	Finds macro expansion	Rewrites register form
Measurement	Reports a number	Includes units/calibration	Analyzes error / uncertainty
Debugging	Tries random changes	Tests wiring and registers	Isolates faults systematically
Collaboration	Follows recipe	Documents reproducibly	Explains and mentors

EnergyTrace inquiry extension

On the MSP-EXP430G2ET only, compare a busy-loop LED program with timer-driven scheduling and low-power idle. Keep LED duty cycle, reporting frequency and supply conditions controlled. Measure average current and total energy using CCS EnergyTrace. Do not claim a power saving before measurement; a one-millisecond wake interval and serial polling may limit the benefit.

9 Programmers: API index

This reference is generated against the shipped v0.2 header set. It inventories the public `EDU_` macro definitions and the `edu_` inline function names. Use the header itself as the definitive source for argument types, register semantics, and current limitations.

Bit and token helpers [edu430_bits.h]

Macro / declaration	Meaning
`EDU_SET(reg, mask)`	Set all mask bits.
`EDU_CLEAR(reg, mask)`	Clear mask bits.
`EDU_TOGGLE(reg, mask)`	Flip mask bits.
`EDU_ALL_SET(reg, mask)`	Test whether all mask bits are high.
`EDU_ANY_SET(reg, mask)`	Test whether any mask bit is high.
`EDU_ALL_CLEAR(reg,mask)`	Test whether all mask bits are low.
`EDU_WRITE_BITS(reg, mask, value)`	Write masked value while preserving other bits.
`EDU_PORT_REG_(port, suffix)`	Internal literal token concatenation.
`EDU_PORT_REG(port, suffix)`	Expand e.g. P1 + OUT to P1OUT.

Digital I/O / interrupts [edu430_gpio.h]

Macro / declaration	Meaning
EDU_GPIO_SELECT_IO(port,mask)	Clear SEL and SEL2 peripheral selections.
EDU_GPIO_OUTPUT(port,mask)	Select GPIO then enable output drive.
EDU_GPIO_OUTPUT_LOW(port,mask)	Latch low before enabling output.
EDU_GPIO_OUTPUT_HIGH(port,mask)	Latch high before enabling output.
EDU_GPIO_INPUT(port,mask)	Select GPIO and make input.
EDU_GPIO_PULLUP(port,mask)	Enable input pull-up through OUT and REN.
EDU_GPIO_PULLDOWN(port,mask)	Enable input pull-down.
EDU_GPIO_FLOAT(port,mask)	Disable internal input resistor.
EDU_PIN_ON(port,mask)	Set output latch bits.
EDU_PIN_OFF(port,mask)	Clear output latch bits.
EDU_PIN_TOGGLE(port,mask)	Toggle output latch bits.
EDU_PIN_HIGH(port,mask)	Read masked input high.
EDU_PIN_LOW(port,mask)	Read masked input low.
EDU_GPIO_IRQ_ENABLE(port,mask)	Enable pin interrupts on P1/P2.
EDU_GPIO_IRQ_DISABLE(port,mask)	Disable pin interrupts on P1/P2.
EDU_GPIO_IRQ_CLEAR(port,mask)	Clear a port interrupt flag.
EDU_GPIO_IRQ_FALLING(port,mask)	Select high-to-low transition.
EDU_GPIO_IRQ_RISING(port,mask)	Select low-to-high transition.

Board-level conveniences [edu430.h]

Macro / declaration	Meaning
EDU_BOARD_LED_A	P1.0 LED mask; color varies by board.
EDU_BOARD_LED_B	P1.6 LED mask; overlaps USCI_B0.
EDU_BOARD_BUTTON	P1.3 active-low switch mask.
EDU_LED_A_ON()	Set P1.0 output latch.
EDU_LED_A_OFF()	Clear P1.0 output latch.
EDU_LED_A_TOGGLE()	XOR P1.0 output latch.
EDU_BUTTON_DOWN()	Test P1.3 input for low state.

Clock / watchdog / sleep [edu430_system.h]

Macro / declaration	Meaning
`EDU_WATCHDOG_STOP()`	Stop watchdog.
`EDU_WATCHDOG_KICK(config)`	Rewrite watchdog control with provided config.
`EDU_IRQ_ON()`	Globally enable interrupts.
`EDU_IRQ_OFF()`	Globally disable interrupts.
`EDU_SLEEP_LPM0()`	Enter LPM0 with interrupts enabled.
`EDU_SLEEP_LPM3()`	Enter LPM3 with interrupts enabled.
`EDU_WAKE_CPU_FROM_ISR()`	Clear CPUOFF upon ISR return.

Function	Meaning
`edu_clock_1mhz()`	Apply factory-calibrated DCO or return failure.

Timer_A [edu430_timer.h]

Macro / declaration	Meaning
`EDU_TIMER_UP(timer,top,clockbits)`	Configure named timer CCR0 in up/interrupt mode.
`EDU_TIMER_STOP(timer)`	Clear timer mode bits.
`EDU_TIMER_READ(timer)`	Read current timer count.
`EDU_TIMER_COMPARE(timer,channel,value)`	Write chosen channel CCR register.
`EDU_TIMER_1MS_TA0()`	Use Timer_A0 at 1 kHz from 1 MHz SMCLK.

Cooperative scheduling [edu430_scheduler.h]

Macro / declaration	Meaning
`EDU_RUN_EVERY(job,now,task)`	Dispatch an elapsed periodic job via edu_job_due.

Function	Meaning
`edu_job_due()`	Test and update due time, handling counter wrap.

ADC10 [edu430_adc.h]

Macro / declaration	Meaning
`EDU_ADC10_RAW()`	Read ADC10MEM raw sample.
`EDU_ADC10_OFF()`	Turn ADC10 conversion/ADC power off.

Function	Meaning
`edu_adc10_init_a4()`	Configure A4 and AVCC/AVSS ADC10 mode.
`edu_adc10_sample()`	Read ADC10MEM after bounded conversion poll.

UART [edu430_uart.h]

Macro / declaration	Meaning
`EDU_UART_RX_READY()`	Test USCI_A0 receive-ready flag.
`EDU_UART_TX_READY()`	Test USCI_A0 transmit-ready flag.

Function	Meaning
`edu_uart_9600_init()`	Map P1.1/P1.2 and set USCI_A0 to 9600 baud.
`edu_uart_put()`	Send one byte with bounded busy poll.
`edu_uart_get()`	Read one byte if a character is available.
`edu_uart_text()`	Write NUL-terminated text byte by byte.

SPI [edu430_spi.h]

Macro / declaration	Meaning
`EDU_SPI_MODE0`	Clock mode 0 bit pattern for TI USCI_B0.
`EDU_SPI_MODE1`	Clock mode 1 bit pattern.
`EDU_SPI_MODE2`	Clock mode 2 bit pattern.
`EDU_SPI_MODE3`	Clock mode 3 bit pattern.

Function	Meaning
`edu_spi_begin()`	Initialize USCI_B0 master with selected mode/divider.
`edu_spi_transfer()`	Exchange one SPI byte; return success/failure.

Hardware I²C [edu430_i2c_hw.h]

Function	Meaning
edu_i2c_begin()	Configure USCI_B0 7-bit master at ~50 kHz.
edu_i2c_wait_start()	Wait for START or NACK with bounded poll.
edu_i2c_wait_tx()	Wait TX-ready or NACK.
edu_i2c_wait_rx()	Wait RX-ready or NACK.
edu_i2c_wait_stop()	Wait for STOP to complete.
edu_i2c_write()	Master write transaction and STOP.
edu_i2c_read()	Master read transaction with STOP handling.
edu_i2c_read_reg8()	Read one byte from an 8-bit register address.

Software I²C [edu430_i2c_soft.h]

Macro / declaration	Meaning
EDU_SOFT_SCL	Software clock mask (P2.0).
EDU_SOFT_SDA	Software data mask (P2.1).

Function	Meaning
edu_si2c_delay()	Software timing gap (not calibrated).
edu_si2c_scl_low()	Pull software SCL low via direction register.
edu_si2c_scl_release()	Release software SCL line.
edu_si2c_sda_low()	Pull software SDA low.
edu_si2c_sda_release()	Release software SDA line.
edu_si2c_scl_high()	Wait for released SCL high / stretching.
edu_si2c_init()	Release both lines and check idle high.
edu_si2c_start()	Generate I²C START condition.
edu_si2c_restart()	Generate repeated START.
edu_si2c_stop()	Generate I²C STOP condition.
edu_si2c_bit()	Clock one outgoing bit.
edu_si2c_write()	Transmit byte and sample ACK.
edu_si2c_read()	Receive byte and generate ACK/NACK.

Function	Meaning
edu_si2c_write_reg8()	Write one 8-bit addressed register.
edu_si2c_read_reg8()	Read one 8-bit addressed register.
edu_si2c_recover()	Nine SCL clocks and STOP bus recovery.

PWM [edu430_pwm.h]

Function	Meaning
edu_pwm_ta0_p16_begin()	Configure TA0/P1.6 at 1 kHz PWM.
edu_pwm_ta0_p16_duty()	Update duty to 1–999 per mille.

Comparator_A+ [edu430_comparator.h]

Macro / declaration	Meaning
EDU_COMP_ON()	Enable Comparator_A+.
EDU_COMP_OFF()	Disable Comparator_A+.
EDU_COMP_OUTPUT_HIGH()	Read comparator result bit.
EDU_COMP_IRQ_ENABLE()	Enable comparator interrupts.
EDU_COMP_IRQ_DISABLE()	Disable comparator interrupts.
EDU_COMP_IRQ_CLEAR()	Clear comparator interrupt flag.

Button events [edu430_button.h]

Function	Meaning
edu_button_reset()	Clear sampled debounce state.
edu_button_pressed()	Emit event after consecutive stable press samples.

Index completeness Inventoried **63 preprocessor definitions** and **38 inline functions** across the shipped headers. The counts include symbolic constants, not only macros that can be called like functions.

10 Teacher-facing capstone projects

Capstone A • A temperature telemetry instrument

Build a 3.3 V-compatible analog temperature probe on A4, acquire ADC samples periodically, convert using the specific sensor transfer function, and report `time_ms,temp_C,status` over UART. Require reference thermometer comparison, calibration discussion, and visible fault detection when readings leave plausible bounds.

Capstone B • A load-cell scale

Use the previously documented HX711 appendices. Implement tare and a known-mass calibration; compare drift and repeatability. Keep the HX711 bit-banging pin driver separate from the general I²C/SPI interface: HX711 is **not** an I²C or SPI peripheral.

Capstone C • Two range sensors

Compare HC-SR04 acoustic transit time with VL6180X short-range optical ranging. Require voltage translation for the 5 V ultrasonic Echo signal. Explain that the VL6180X needs a two-byte register address and device-specific initialization, so the v0.2 I²C reg8 helper alone is insufficient. Treat these as different measurement models, not interchangeable numbers.

Capstone D • Scheduled instrument and power

Use timer-driven cooperative tasks to sample a sensor, update an LED state, log data, and respond to the button. Record task periods and maximum run times. Where a G2ET is available, use EnergyTrace to compare active idle and LPM0. Ask students to document whether the scheduler meets the timing needs of each sensor.

11 Testing, qualification and release engineering

What has been checked

Level	Evidence in v0.2	Interpretation
Static/host	Eight demonstration sources compiled in mock MSP430 host syntax checks.	Useful for C consistency, not target hardware correctness.
Deterministic logic	Three tests cover simple GPIO, button and scheduling behaviors.	Constrained logic checks, not timing proof.
CCS actual target	ProjectSpec candidates provided; import and target build still to be run.	Do not assume any particular CCS release has accepted them.
Electrical / timing	No physical bus/scope/EnergyTrace measurements in authoring environment.	Protocol and hardware behavior remain provisional.
Long-run / errata	Not yet qualified.	Cannot claim production readiness.

Recommended validation plan

01 — Compile matrix

Compile each of eight examples in installed CCS for MSP430G2553; log compiler version, flags, warnings, flash/RAM and ProjectSpec import behavior.

02 — Board matrix

Run onboard-only examples on original G2 and G2ET, documenting GPIO behavior, P1.3 button and UART backchannel.

03 — UART and clock

Measure UART baud or logic timing; verify 9600 at calibrated 1 MHz and record expected DCO drift.

04 — SPI smoke and failure

Use a known 3.3 V SPI memory, read verified JEDEC ID, inspect waveform, force CS and mode errors.

05 — I²C transactions

Use a known slave, test address ACK, 1 and N byte reads, repeated START, STOP timing, disconnect and SDA stuck-low.

06 — Scheduler and power

Compare task timing under worst-case UART load, rollover and LPM0; if available, use EnergyTrace with repeatable conditions.

Release gating

Move a module from experimental to bench-qualified only after a reproducible firmware image, a wiring diagram, captured transaction trace, actual observed outputs, and a failure-mode test are committed. Keep the GPL-3.0 source, TI references, and attribution alongside the firmware.

12 Troubleshooting and field checklist

Observation	First checks	Likely class of issue
CCS will not program target	Device variant, eZ-FET vs legacy debugger, jumpers, target VCC	Toolchain / wiring
No LED change	P1DIR, P1OUT, pin mux, LED jumper	GPIO configuration
UART text is garbage	SMCLK, UCA0BR0, 9600 terminal, jumper orientation	Clock / serial divider
UART port absent	USB drivers, cable, board debug connection	USB virtual COM driver
I²C no ACK	3.3 V pull-ups, common ground, 7-bit address, pin mux	Hardware / protocol
I²C hangs	SDA/SCL stuck, clock stretching, missing STOP, incomplete transfer	Bus state / timeout
SPI unexpected ID	CPOL/CPHA, CS, pin direction and read-frame length	Transaction framing
ADC reading drifts	VCC, input impedance, calibration, sensor grounding	Analog measurement
Two tasks run late	Blocking UART, long I²C sensor timeout, shared Timer_A	Scheduler overrun
LPM3 stops scheduler	SMCLK unavailable in LPM3 under supplied setup	Clock-power mismatch

13 Source lineage, further reading and glossary

This manual documents the existing RandoSY Edu430 FwLib v0.2 distribution and prior MSP430 Macro Evolution Manual. The API philosophy was influenced by Piconomix PX-FwLib, but its source is not copied here. The course collection is GPL-3.0 due to its adaptation lineage; the upstream Piconomix work is MIT licensed.

Primary references

- Edu430 FwLib repository — library sources, tests, CCS examples.
- Piconomix PX-FwLib — architecture and education inspiration.
- TI MSP430G2553 product resources — datasheet, errata and device information.
- TI MSP430x2xx Family User Guide (SLAU144) — register and peripheral operation.
- TI MSP-EXP430G2ET LaunchPad User Guide (SLAU772) — board, UART, debugger and EnergyTrace.
- ADolbyB MSP430G2553 Labs — historical example lineage (GPLv3).

Glossary

Term	Meaning
Bit mask	Bits selected for a register operation.
GPIO	General-purpose input/output.
Register	Memory-mapped peripheral control or data location.
ISR	Interrupt service routine.
UART	Asynchronous serial interface.
SPI	Synchronous serial bus with explicit selection.
I²C	Two-wire open-drain addressed communication bus.
ACK / NACK	Acknowledgment / no acknowledgment in a bus transaction.
ADC	Analog-to-digital converter.
PWM	Pulse-width modulation; varying duty cycle.
LPM0 / LPM3	MSP430 low-power modes with differing clock availability.
Cooperative scheduler	Main-loop functions that return voluntarily, dispatched by elapsed time.
EnergyTrace	TI tool to measure target energy/current using suitable debug probe.

Closing design principle

From bits to behavior A successful learner should be able to move in either direction: from a scientific question to readable firmware, and from a readable firmware call to the exact bits, peripherals, timings and voltages that make it true.